Storage en K8S

CAS TRAINING

BUENAS PRACTICAS STORAGE

- Persistent Volumes (PV)
- Persistent Volume Claims (PVC)
- Modos de acceso (Access Modes)

Docker

- /var/lib/docker
  - Overlay2
  - Containers
  - Images
  - Volumes

```
# Dockerfile for Application 1
FROM ubuntu

RUN apt-get update && apt-get -y install python
RUN pip install flask flask-mysql

COPY . /opt/source-code

ENTRYPOINT FLASK_APP=/opt/source-code/app.py flask run
```

docker build Dockerfile -t mmumshad/my-custom-app

```
# Dockerfile2 for Application 2
FROM ubuntu
RUN apt-get update && apt-get -y install python
RUN pip install flask flask-mysql
COPY app2.py /opt/source-code
ENTRYPOINT FLASK_APP=/opt/source-code/app2.py flask run
```

docker build Dockerfile2 -t mmumshad/my-custom-app-2

COPY-ON-WRITE

docker run mmumshad/my-custom-app

Volumen mounts (Volúmenes)

- /var/lib/docker/volumes

```
docker volume create data_volume
docker run -v data_volume:/var/lib/mysql mysql
```

Bind Mounts

- Usar un directorio específico del host
- docker run -v /data/mysql:/var/lib/mysql mysql

```
docker run \
  --mount type=bind,source=/data/mysql,target=/var/lib/mysql \
  mysql
```

Storage drivers

- AUFS
- ZFS
- BTRFS
- Device Mapper
- Overlay
- Overlay2

STORAGE DRIVERS

AUFS | ZFS | BTRFS | DEVICE MAPPER | OVERLAY

VOLUME DRIVERS

Local | Azure File Storage | Convoy | DigitalOcean Block Storage | Flocker | gce-docker | GlusterFS | NetApp | RexRay | Portworx | VMware vSpher

https://docs.docker.com/engine/extend/legacy_plugins/

```
docker run -it \
  --name mysql \
  --volume-driver rexray/ebs \
  --mount src=ebs-vol,target=/var/lib/mysql \
  mysql
```

CONTAINER
STORAGE
INTERFACE

portworx

Amazon
EBS

DELL
EMC

RPC

CreateVolume

DeleteVolume

ControllerPublishVolume

- ✓ SHOULD call to provision a new volume
- ✓ SHOULD call to delete a volume
- ✓ SHOULD Call to place a workload that uses the volume onto a node.

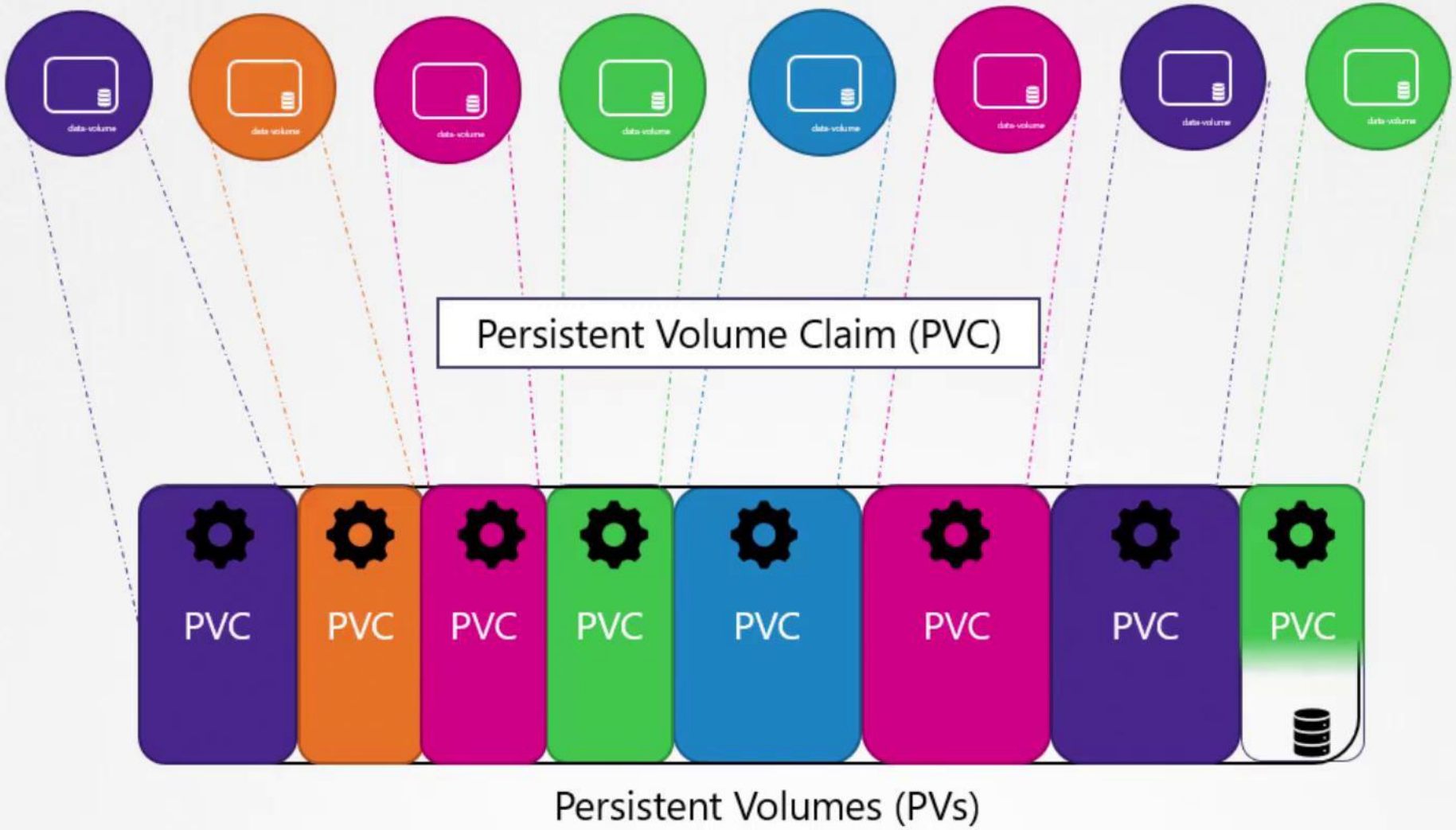
- ✓ SHOULD provision a new volume on the storage
- ✓ SHOULD decommission a volume
- ✓ SHOULD make the volume available on a node

https://github.com/container-storage-interface/sp

Volúmenes

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: random-number-generator
spec:
  containers:
  - image: alpine
    name: alpine
    command: ["/bin/sh", "-c"]
    args: ["shuf -i 0-100 -n 1 >> /opt/number.out;"]
    volumeMounts:
    - mountPath: /opt
      name: data-volume
  volumes:
  - name: data-volume
    hostPath:
      path: /data
      type: Directory
```

Persistent Volume

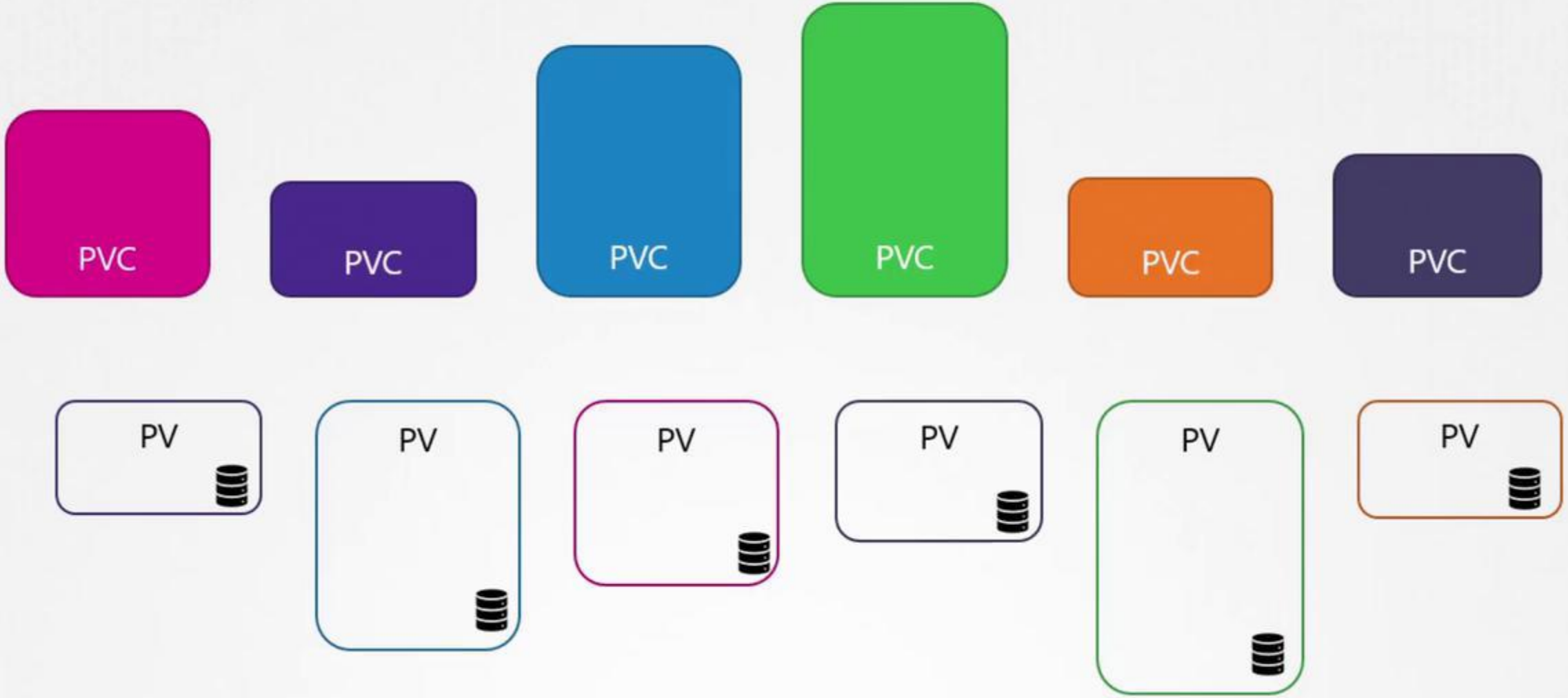


```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-vol1
spec:
  accessModes:
    - ReadWriteOnce
  capacity:
    storage: 1Gi
  hostPath:
    path: /tmp/data
```

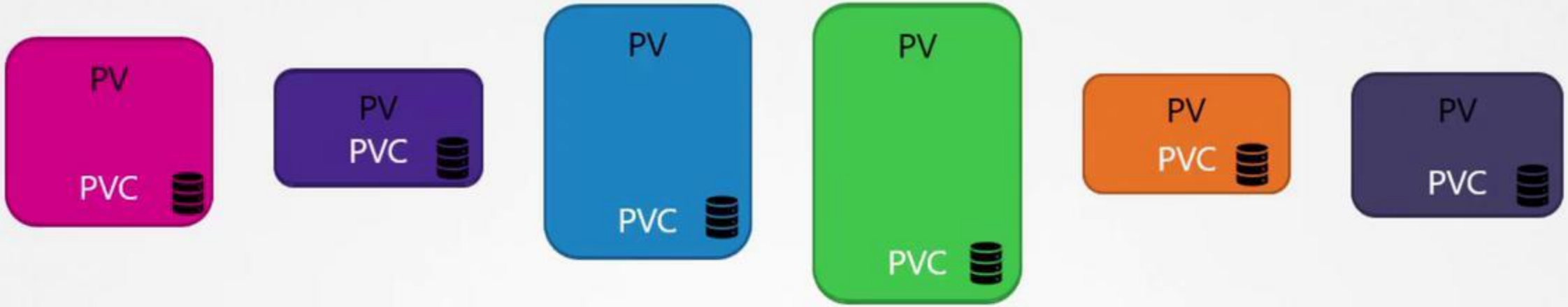
kubectl create -f pv-d.yml

- 🟢 **ReadWriteOnce (RWO)**
  → Lectura/escritura por **un solo nodo**
- 🔵 **ReadOnlyMany (ROX)**
  → Solo lectura por **varios nodos**
- 🟣 **ReadWriteMany (RWX)**
  → Lectura/escritura por **varios nodos**

Persistent Volume Claim



Binding



Sufficient Capacity

Access Modes

Volume Modes

Storage Class

Binding

PVC

```
selector:
  matchLabels:
    name: my-p
```

PV

```
labels:
  name: my-pv
```

PV

Sufficient Capacity | Access Modes | Volume Modes | Storage Class | Selector

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: myclaim
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 500Mi
```

```
kubectl get persistentvolumeclaim
NAME      STATUS   VOLUME   CAPACITY   ACCESS MODES
myclaim   Pending
```

kubectl create -f pvc-definition.yaml

kubectl delete persistentvolumeclaim myclaim

Storage class

- Permiten el aprovisionamiento dinámico.
- Con Storage classes puedes aprovisionar automáticamente:
  - AWS EBS
  - Azure Disk
  - Google Persistent Disk
  - OCI Block volume
- Las Storage Classes eliminan la necesidad de crear PVs manualmente.

```bash
gcloud beta compute disks create \
  --size 1GB \
  --region us-east1 \
  pd-disk
```

```yaml
# pv-definition.yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-vol1
spec:
  accessModes:
    - ReadWriteOnce
  capacity:
    storage: 500Mi
  gcePersistentDisk:
    pdName: pd-disk
    fsType: ext4
```

```yaml
# pvc-definition.yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: myclaim
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 500Mi
```

```yaml
# pod-definition.yaml
apiVersion: v1
kind: Pod
metadata:
  name: random-number-generator
spec:
  containers:
    - image: alpine
      name: alpine
      command: ["/bin/sh", "-c"]
      args: ["shuf -i 0-100 -n 1 >> /opt/number.out;"]
      volumeMounts:
        - mountPath: /opt
          name: data-volume
  volumes:
    - name: data-volume
      persistentVolumeClaim:
        claimName: myclaim
```

```yaml
# sc-definition.yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: google-storage
provisioner: kubernetes.io/gce-pd
```

```yaml
# pvc-definition.yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: myclaim
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: google-storage
  resources:
    requests:
      storage: 500Mi
```

```yaml
# pod-definition.yaml
apiVersion: v1
kind: Pod
metadata:
  name: random-number-generator
spec:
  containers:
    - image: alpine
      name: alpine
      command: ["/bin/sh", "-c"]
      args: ["shuf -i 0-100 -n 1 >> /opt/"]
      volumeMounts:
        - mountPath: /opt
          name: data-volume
  volumes:
    - name: data-volume
      persistentVolumeClaim:
        claimName: myclaim
```

```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-voll
spec:
  accessModes:
    - ReadWriteOnce
  capacity:
    storage: 1Gi
  awsElasticBlockStore:
    volumeID: <volume-id>
    fsType: ext4
```

Volume storage options

- NFS
- GlusterFS
- Flocker
- Fibre Channel
- CephFS
- ScaleIO
- Public cloud storage solutions like AWS EBS, Azure Disk or File, and Google Persistent Disk